

Hybrid Resonator+IR Percussion Synthesizer – Design Brief

Background: Nonlinear Percussion Dynamics and Hybrid Modeling

Gong- and cymbal-like instruments exhibit extremely **nonlinear vibration behaviors** under strong excitation. Small taps produce relatively “bell-like” inharmonic tones, but hard strikes cause dramatic effects: pitch shifts, rapid growth of high-frequency energy, subharmonic tones, and even chaotic noise components ¹. In other words, as the strike intensity increases, the response transitions from *linear* (small, harmonic-like vibrations) to *inharmonic* (slight detuning of modes) and eventually to *chaotic* regimes with turbulent, noisy vibrations ². This **wave turbulence** phenomenon results in the instrument’s energy spectrum “diffusing” from low to high frequencies over time, contributing to the characteristic shimmering or “glinting” sound of cymbals and gongs ³. Accurate physical simulations of these nonlinear plate vibrations are computationally intensive, often requiring hundreds or thousands of modes to capture a cymbal’s dense spectrum ⁴. For example, a simple cowbell can be modeled with a few dozen resonant modes, but a realistic crash cymbal “wash” may need *hundreds or even thousands* of modes ⁴ – something impractical for real-time CPU synthesis.

Hybrid approach: To achieve rich sound efficiently, we combine **modal resonators** with a **convolution-based impulse response (IR)**. Prior work in hybrid percussion modeling shows that feeding an excitation signal through **banks of resonant filters or waveguides** can capture an instrument’s modal character, while **convolving** with a recorded impulse response reproduces the complex resonant “tail” ⁵. Roberto Aimi’s convolution percussion system, for example, used the audio of a drum hit (or an electronic trigger) directly convolved with a stored instrument IR to “replace” the physical resonance with any desired response ⁶. This technique treats the convolution processor as the resonator, allowing a damped input tap to excite a rich, long IR ⁷. Our design follows a similar philosophy: a set of digital resonators will model the strongest modal frequencies and nonlinear interactions in real time, while a long IR (e.g. recorded from a gong or plate) imparts the detailed acoustic character and lengthy decay that would be costly to simulate directly. By carefully shaping the excitation and coupling energy into the resonators, we **preserve the instrument’s dynamic nuances** (e.g. accumulated vibration from repeated strikes, brightness increasing with energy, chaotic components at extreme intensities) without an overly complex physical model. This document outlines the system architecture in layers – from input triggers and energy accumulation to excitation synthesis, resonators, convolution processing, and the parameter controls that tie them together – as a **JUCE** plugin design.

Strike Event Detection Layer (Hit/Event Input)

This layer produces clean, consistent “**strike events**” from either audio transients or MIDI triggers. Its job is to detect and characterize each percussive hit so downstream stages can respond predictably:

- **Audio Tap Detection:** For an incoming audio signal (e.g. from a contact mic or trigger pad), we implement an onset detector. A simple approach is using a **peak detector with threshold and hold:**

when the signal's short-term amplitude or filtered energy exceeds a set threshold, register a strike event. To avoid double-triggering on one hit, use a brief **retrigger lockout** (e.g. 50–100 ms) after a trigger. The system should capture the **timing and intensity** of the tap. Intensity can be estimated from the peak amplitude or envelope of the input pulse. For reliability, a band-pass filter around expected hit frequencies (or a transient detector) can improve discrimination of genuine strikes versus noise. In JUCE, one could use the `dsp::EnvelopeFollower` or custom code in `processBlock` to measure incoming audio and generate a MIDI-like event when a threshold is crossed.

- **MIDI Integration:** MIDI note or pad triggers are handled similarly – each note-on (or drum pad hit) is treated as a strike event with an associated velocity. We map MIDI velocity to an initial energy value. The goal is to **unify audio and MIDI hits** so that whether the user taps a mic or plays a MIDI pad, the subsequent layers receive a normalized *StrikeEvent(time, velocity)*. This may involve scaling and calibrating the audio-derived velocity to match the MIDI 0–127 range or an internal 0.0–1.0 energy scale.
- **Consistency and Jitter Handling:** The layer can apply slight smoothing to the detected velocities to avoid unnatural jumps. For example, if the user's input velocities are highly variable, an optional **velocity curve** or smoothing filter can produce more consistent perceived loudness. Timing-wise, events should be timestamped (if using a high-level framework) or simply processed immediately (since this is a real-time system). The design assumes latency must be minimal – using a lookahead onset detector (FFT-based) is likely unnecessary for this application, as responsiveness is paramount.

Output of this layer: a stream of discrete strike events with parameters like **timing, intensity**, and possibly **strike type**. (All strikes are treated similarly for now, but in future an event might include an estimate of whether it's a fingertip tap, mallet, etc., if derived from audio – this could route to different excitation profiles. For now, we use a single unified excitation model for all strikes.)

Energy Accumulation Layer (Global & Banded Reservoirs)

Once a strike event occurs, we **inject energy** into the system's reservoirs to simulate how repeated hits accumulate vibrational energy. Instead of each hit producing an independent sound that decays from silence, this layer ensures the instrument's "stored" energy carries over between hits – just like a real cymbal continues vibrating from previous strikes ⁸. We use two levels of energy tracking:

- **Global Energy Reservoir:** This is a single scalar representing the overall energy in the instrument at a given time. It rises when strikes occur and gradually decays when no new energy is added. We can implement this as a **leaky integrator**: on each strike, add a certain amount (proportional to strike velocity) to the reservoir; every sample or block, multiply the reservoir by a decay factor `< 1` to simulate energy loss. The decay time constant (e.g. a few seconds) corresponds to how long the instrument rings when excited. A real gong can sustain energy for tens of seconds, but we may choose a somewhat shorter default for practicality (adjustable via parameters). Crucially, this reservoir does **not reset to zero on each hit** – a second hit shortly after the first will build on the remaining energy ⁸. This models how a cymbal hit "goes from decaying vibration to new vibration, rather than from silence each time" ⁸.

- **Banded Energy Reservoirs:** In addition to the global energy, we maintain separate energy levels for different **frequency bands or modes**. For example, we can have 8 band-specific energy values, one per resonator (mode). These “banded reservoirs” let us track how energy is distributed across low, mid, high frequency modes. A strike can inject energy unequally – e.g. a soft mallet might favor low-frequency modes, whereas a hard stick adds more high-frequency energy. By maintaining per-band energy, we enable **frequency-dependent responses** (like high-frequency components decaying faster or saturating sooner than low frequencies, reflecting physical damping differences ¹). Each band reservoir similarly decays over time at its own rate (configurable per band, since higher modes often have higher damping).
- **Nonlinear Energy Injection:** We introduce a **nonlinear mapping** for how strike intensity adds to the reservoirs. Rather than simply summing energies (which could be linear and unbounded), we use a saturating or compressive formula. For example, if `E_global` is the current global energy, a new strike of normalized strength `S` could update it as:

$$E_global = \text{clamp_max}(E_global + k * S^p * (1 - E_global/E_max), E_max)$$
Here `E_max` is a chosen saturation ceiling (to prevent unrealistically infinite energy), `p` is an exponent (<1 for compression) and `k` is a scaling gain. This formula means that when `E_global` is low, a strike adds energy roughly proportional to `Sp`; but when the system already has high energy (`E_global` near saturation), the **increment is reduced** (diminishing returns). The effect is that **loud hits on an already ringing instrument won't linearly double the volume** – instead, they nudge the system closer to a “saturated” loudness. This models the physical fact that an instrument can only sustain so much vibrational amplitude before nonlinear losses increase (and our downstream model will channel excess energy into detuning and noise rather than unlimited amplitude). For banded reservoirs, a similar injection happens per band: each gets a fraction of the strike energy, potentially with its own saturation. For instance, if a particular mode already contains high energy, additional hits might **not linearly increase that mode's amplitude** – instead, extra energy might spill into other behaviors (we'll handle that via mode coupling and noise in later stages).
- **Decay and Release:** All reservoirs leak energy over time. The global reservoir decay corresponds to the overall **ring-down** of the instrument's sound. Banded reservoirs can have shorter decay for higher bands to reflect that high frequencies damp out sooner (the **loss modeling** noted as crucial for realistic energy migration in cymbal crashes ⁹). We can use simple first-order decay: $E = E * \exp(-dt/\tau)$ per time step, or an equivalent per-sample multiplier. These time constants (τ) are key parameters: e.g. low-frequency modes might $\tau \approx 5\text{--}10$ seconds, mid frequencies 3–5 s, highest frequencies 1–2 s, giving a long overall sustain but a natural darkening over time as highs fade.
- **Output of layer:** continuous values for `E_global` and each `E_band[i]` at audio rate or control rate. These will influence the next stages – for instance, they can modulate resonator amplitudes, frequency shifts, and noise injection. We may also derive a quick **energy envelope** (like a boolean or fast ramp) when a new strike occurs, to shape the *immediate* excitation burst (next layer). The **nonlinear injection plus saturation** ensures that after many rapid hits, the energy levels tend toward a steady-state limit (preventing runaway feedback), while still preserving the effect of accumulating intensity.

Excitation Synthesis for Repeated Stick Strikes

This stage generates the **audio excitation signal** that feeds the resonators and convolution. It must produce a credible *strike sound* for each hit, yet adapt when hits repeat in quick succession on an already vibrating surface. We design a **dual-path excitation**: a primary transient burst for the percussive attack, and a secondary component that accounts for the existing energy in the system.

- **Primary Attack Burst:** Every strike triggers a short burst of sound representing the **stick-on-instrument impact**. This could be synthesized as a burst of filtered noise or a short wavetable. We want a controllable spectrum – e.g. a softer mallet yields a mellower, low-passed noise thump, whereas a hard stick gives a bright, wideband click. This is controlled by a **“Mallet Hardness”** parameter that can adjust a filter on the noise (e.g. a low-pass cutoff or tilt EQ on the burst). Duration-wise, this burst might be only a few milliseconds to ~50 ms – enough to excite all resonators. We may shape it with an amplitude envelope (fast attack, short decay) to avoid a DC offset and give it a percussive shape. The amplitude of this burst is scaled by the strike velocity and possibly moderated by current energy (for example, if the instrument is *very* energized, the marginal impact might be perceived as slightly less pronounced in isolation, but this is subtle; we mostly let the resonators handle the difference).
- **Secondary “Energy-Driven” Excitation:** This is a dynamic component that injects additional sound depending on the **current energy state**. As the instrument rings and especially when it’s in a highly excited (near-saturated) state, real gongs start to produce a **rushing, chaotic noise** alongside the tonal ringing ¹. We mimic this by introducing a **noise excitation proportional to E_{global}** . Practically, this could be implemented as a continuous low-level noise source that is gated by the energy envelope – for example, white noise passed through a band-pass or high-pass filter (to emphasize the shimmery highs), scaled by some function of E_{global} . When a new strike occurs on an already loud cymbal, not only do we hear the stick attack, but the instrument’s existing vibration can cause a burst of noisy “hash” or a broadening of the spectrum ¹⁰ ¹. In our model, we can transiently increase the level of this noise component right after a strike if E_{global} is above a threshold. Essentially, **high energy enables a secondary excitation**: a brief **“wash” of broadband noise** that reinforces the chaotic character. This could be realized by tapping the global reservoir’s rise – e.g., whenever E_{global} increases quickly (a hit), generate a short noise burst whose amplitude is proportional to the *current* energy (so hitting an already loud cymbal yields a stronger noise burst than hitting a quiet one). This reflects the concept of **mode coupling into a continuum of high-frequency modes** – instead of explicitly simulating hundreds of modes, we approximate their collective effect as filtered noise. The noise burst is also subject to its own decay (very fast, maybe 100–200 ms) so that it simulates that *chhh* sound right after a crash.
- **Repeated Strike Adaptation:** With these two components, the system naturally adapts to repeated hits. The first hit (from silence) produces a clean attack burst and little secondary noise (since E_{global} was low, no chaotic wash yet). As energy accumulates, subsequent hits produce just as strong an attack burst (to ensure articulation), **but now the background noise level and overall resonance are already elevated**. The new attack riding on top of an existing sustain will sound different than an isolated hit – our model doesn’t stop the ongoing resonator vibrations (they continue from the previous hit’s energy) and adds the new excitation on top. Thus, the resonators will already be in motion (phase may not align perfectly, which is fine and realistic, causing some interference or beating). We do not attempt to restart phases; we simply *add* new excitation forces to

the resonator inputs. This ensures the **sound “builds up”** like a real cymbal roll or repeated crash, rather than retriggering the same sample from zero ⁸. If anything, we might slightly taper the amplitude of the primary burst when `E_global` is very high to avoid unrealistically loud transients on top of a loud sustain – this can be handled by the saturating energy injection mentioned earlier (the burst itself is proportional to the unsaturated added energy).

- **Implementation:** In JUCE, this excitation can be generated per event. For example, we can prepare a short buffer for the burst: fill with noise, apply an envelope and filter. For continuous noise tied to energy, we might generate noise each audio callback and multiply it by a factor derived from `E_global` (which is updated every sample). Alternatively, we could convolve a tiny noise burst with a short FIR tuned for desired spectral tilt to get a consistent burst shape. Low-latency is key, so all processing is time-domain and straightforward. We should also consider **anti-aliasing** if the noise burst is very bright (perhaps band-limit the noise to Nyquist or slightly below).
- **Exciting Individual Bands:** Optionally, the excitation burst can be **band-split** so that different resonators get different proportions. A realistic approach is to excite all modes according to their modal participation factors – in practice, we might predefine a gain for each of the 8 resonators that determines how much of the burst goes into that resonator. For instance, if the burst is generated as white noise, passing it through band-pass filters centered at each modal frequency and feeding those into each resonator (instead of the same broadband burst to all) can yield a more authentic excitation (each mode “feels” a force in its frequency range). A simpler approach is to have a **frequency-dependent amplitude**: e.g. the higher-frequency resonators might receive relatively less energy from a soft hit and more from a hard hit. We can map strike velocity to an *excitation spectrum tilt*. This detail can be refined experimentally and exposed in the advanced parameters (e.g. a table of per-resonator excitation gains or a filter controlling excitation color).

In summary, this layer produces the **excitation audio** that will drive both the resonators and the convolution “body” response. It ensures **articulation for each hit** while honoring the presence of existing vibrations through an energy-dependent noise reinforcement. The result should be a lively, responsive excitation that makes single hits clean and multiple hits accumulate into a lush, sustained crash.

Resonator Integration (Modal Resonator Bank with Nonlinear Dynamics)

At the core of the synthesis, we use **8 resonators** to simulate the instrument’s principal modes of vibration. Each resonator can be thought of as a band-limited oscillator or filter that rings at a specific frequency (the mode frequency) with a certain decay (damping) and gain. This section describes how we configure and connect these resonators, including **detuning behavior**, **mode coupling**, and how they are influenced by the energy reservoirs.

- **Modal Filter Design:** We choose a resonator implementation that is **high-Q and low CPU**. In JUCE, a straightforward option is 2nd-order bandpass or resonant filters tuned to each mode frequency (with parameters for frequency, Q, and gain). Alternatively, a modal oscillator algorithm (like a digital resonator or “very-high-Q phasor filter” ¹¹ ¹²) could give stable ringing without frequency warping. For clarity, we can start with biquad bandpass filters: each resonator `i` has frequency `f_i`, damping (bandwidth) related to its decay time, and an input gain `g_i`. The filters are excited

by the **excitation signal**: effectively we feed the same excitation burst into all resonators (or a filtered version per above). Each filter then outputs a decaying sinusoidal ringing at its f_i . By summing all 8 outputs, we get the composite modal sound. We will set the mode frequencies to reflect an inharmonic series typical of a gong or cymbal. (If we were modeling a specific instrument, these could come from analysis or physical calcs; for design, we might pick frequencies that are inharmonic ratios, e.g. 1x, 2.3x, 2.9x, 4.1x, etc., of a base frequency, to simulate a complex spectrum.)

- **Detune Spread Driven by Energy:** To emulate the *inharmonic shift* that occurs at high amplitudes, the resonators will **detune slightly as a function of the energy**. In real nonlinear plates, modal frequencies can shift (usually downward) when vibration amplitude is high, due to nonlinear softening ¹. We incorporate this by adjusting each resonator's frequency based on the global (or per-band) energy. For example, we can define each mode's frequency as: $f_{i_actual} = f_i * (1 - D_i * E_{global}^n)$, where D_i is a detune depth constant and n might shape the curve (e.g. squared or so for smoothness). At low E , $f_{i_actual} \approx f_i$; at high E , the frequency drops slightly (if D_i is positive). Alternatively, some modes might shift up if desired (setting D negative). We can have an **inharmonic amount knob** to scale all D_i simultaneously (this would be the "Chaos" or "Nonlinearity" parameter from the user's perspective). Another approach is to prepare two sets of mode frequencies: one for linear regime, another for a high-energy regime (perhaps derived from measurements or simply a slightly more inharmonic set), and then **morph between them** as E_{global} rises. This is akin to the strategy used by Skare and Abel (2019) where they identified modes present only at high velocities and introduced them when energy crossed a threshold ¹³ ¹⁴. In our simpler implementation, continuously detuning the modes as a function of energy can already give a sense of the sound "spreading" and becoming more complex as loudness increases. The detuning amount per mode (D_i) could be higher for the higher-frequency modes (since those often shift more dramatically ¹). By default we might allow up to a few percent frequency shift at maximum energy. This **energy-dependent detune** gives that inharmonic "bending" effect during loud crashes.

- **Resonator Amplitude and Damping Modulation:** Each mode's amplitude naturally follows from excitation and its damping. However, we can further modulate amplitude or decay based on energy to simulate nonlinear damping. For instance, at very high amplitude, a mode might damp a bit faster (due to increased air resistance or internal losses). We could achieve this by making the effective Q factor drop slightly when its band's energy is high. The banded energy reservoir $E_{band[i]}$ can control this: if $E_{band[i]}$ is above some threshold, reduce the Q (increase damping) for resonator i . This would make loud crashes somewhat "wash out" faster in the highs, which aligns with how sustained loud vibrations tend to generate more noise and less pure tone over time ¹ ¹⁵. This is an advanced nuance; initial implementation can hold Q constant per mode (set from a nominal decay time), since the convolution tail will also contribute to the damping perception.

- **Mode Coupling (Optional Feedback Bus):** To imitate the **energy transfer between modes** that occurs in nonlinear plates ¹⁶ ¹³, we include an **optional coupling network**. If enabled, the resonators will feed small amounts of energy into each other depending on their amplitudes. A simple scheme is: whenever a resonator's output is large (indicating that mode is strongly excited), a fraction of that signal is **fed into the input of one or more other resonators**. This can be done via a predefined coupling matrix $\{\psi_{mn}\}$ where, for example, if mode 1 (high frequency) is loud, it injects some energy into mode 2 (lower frequency) causing mode 2 to start ringing too (subharmonic generation). Skare et al. implemented this by immediately scaling down one mode's

output and adding it into another mode's excitation term whenever an energy threshold was exceeded ¹³ ¹⁴. We can approximate that: if `E_global` or a specific `E_band` crosses a threshold, trigger a transfer: e.g. take 10% of mode m's current amplitude and add to mode n's state. To avoid phase issues, one can transfer at a zero-crossing or just allow a slight pop (which might even be masked by the chaotic sound). The *coupling bus* can be structured as a small summing network updated each sample or block – e.g. `in_n += k * out_m` for coupled pairs (m→n) when conditions are met. This effectively **trades energy** between modes and can make new frequencies “emerge” during a loud vibration ¹³ ¹⁷. We will expose a **Coupling Amount** parameter (k) that scales all ψ_{mn} . Setting $k = 0$ gives independent resonators (linear modal synthesis). Increasing k yields more interactive behavior: for instance, a strong high mode might bleed into a low mode causing a beating or intermittent “wow” sound, similar to how a real cymbal crash may suddenly favor a lower roar as energy redistributes ¹⁶. The coupling network must be used carefully to avoid instability (hence optional). We will implement safety clamps: e.g. do not let any single coupling transfer exceed a small fraction of target mode's current energy, and possibly disable coupling when overall energy is extremely high to prevent runaway. In practice, this coupling adds a lot of realism for chaotic regime, so it's a valuable advanced feature.

- **Resonator Output Mixing:** We have 8 resonator outputs which we sum to produce the “modal synthesis output.” In a stereo context, we might pan these modes across the stereo field for a richer image (for example, slightly spread frequencies L/R or use random phase differences). The **voice mix** can refer to blending the direct resonator output with other elements. It might also refer to layering multiple sets of resonators for complexity. However, given CPU constraints, we stick to one set of 8. Instead, the “**voice mix**” concept in our design will be handled by how we mix resonators vs convolution (see next section) and possibly blending the resonators' linear vs detuned state. For instance, at low energies the output is mostly the clean resonator tones; at high energies, the detuned frequencies and noise make up more of the sound (effectively mixing “voices”: the instrument's orderly voice vs its chaotic voice). We can formalize that by crossfading between the nominal resonator output and a version that's broadened (or even duplicated and detuned slightly). Another interpretation: we could duplicate each resonator into two voices that are a few Hz apart in tuning (to simulate slight splitting of modes due to asymmetry). These two voices per mode could be mixed with an amount that increases with energy to simulate *mode splitting*. That might be too heavy for 8 modes (16 oscillators), but could be an advanced option.

In summary, the resonator bank provides the **structured, inharmonic tonal content** of the percussion sound. With 8 modes, we capture the major frequency components efficiently. Nonlinear enhancements like energy-driven detuning and coupling allow the modal synthesis to go beyond a static linear sound, introducing the “**bending**” and “**warble**” characteristic of loud gongs ¹. While 8 modes is far fewer than a real cymbal's spectrum, the addition of the convolution tail (next section) and noise/coupling means the **perceived density of sound** will be much higher than 8 pure tones – the modes give a scaffold of identifiable frequencies, and the other elements fill in the complexity. Notably, using only a handful of modes keeps the CPU usage low (JUCE can easily handle 8 biquad filters). If desired, this design could be extended to more modes or even use a GPU for hundreds of modes ¹⁸ ¹⁹, but we assume 8 is a practical compromise for a real-time plugin.

Convolution “Body” Response and Character Control

Parallel to the resonator bank, we include a **convolutional reverberation** component that provides the extended impulse response of the instrument’s body or acoustic environment. This convolution uses a **long IR (impulse response)**, on the order of 1–3 seconds or more, to reproduce the fine details of the percussion’s sound that are not captured by the 8 resonators. It effectively acts as a complex resonator with *extremely many modes* (since an IR can encode a very dense modal response). This section covers how we integrate the convolution and how we give the user control over its tonal character:

- **Routing and IR Selection:** The convolution is fed by an **excitation signal** just like the resonators. There are a few options for what to feed into the convolution engine:
 - A straightforward approach: feed the *same excitation burst* (from the Excitation Synthesis layer) into the convolution. This means the IR will be excited by a short click or noise that represents the stick hit. If the IR is a recording of a gong’s impulse response (like hitting a real gong with a similar excitation and capturing the output), then convolving our synthetic excitation with that IR will produce a very realistic gong sound (including all its modes) ⁶. However, since we are also generating resonator output in parallel, we might *double up* on some frequencies. To avoid phasing issues, we could ensure the IR is more focused on the **high-frequency “wash”** and long tail, while our explicit resonators cover the strongest low/mid partials. If we have the ability, we could derive the IR from a recording with those main modes notched out (leaving mostly residual and high-frequency content). In practice, an easier method is to just accept some overlap and potentially **weight the mix** (e.g. reduce resonator gain in bands that the convolution already strongly covers).
 - Another approach: feed the **resonator output** (or a mix of it) into the convolution as the excitation. This would effectively convolve the already modal sound with the IR, potentially simulating an “acoustic space” around it. This is more like using convolution as a *reverb*. But the user prompt suggests using convolution as part of the instrument body itself, so likely the IR is of the instrument (not a room). Thus, convolving resonator output with an instrument IR would be like double-filtering the modes – not ideal.
- Therefore, we choose the **first approach**: feed the raw strike excitation into the convolution. The convolution output will then represent the **full instrument response** to that strike. Meanwhile, we also have the resonator output representing a simplified model of the same. To combine them, we can **mix the convolution output with the resonator output**. Essentially, the resonators give a clean, controllable modal foundation, and the convolution adds *richness and realism* (many more subtle modes, rattles, and the long sustain). The mix between these can be adjusted by a **“Resonators vs IR Mix”** parameter (or simply ensure they complement each other without overwhelming).
- **Pre-EQ (Excitation Filtering):** We provide controls to shape the tone **before** the convolution, i.e. filtering the excitation going into the IR. This is powerful because convolving is linear: if you EQ the input, it’s equivalent to EQing the output in many ways. By adjusting the input spectrum, the user can tailor how the IR is excited. For example, if the IR is of a very bright cymbal but the user wants a darker tone, a pre-EQ can roll off highs from the excitation, resulting in less high-frequency content excited in the IR response. We can implement two simple pre-filters: a **low-shelf** and **high-shelf** (or a single **tilt filter**). A **“Tone”** knob could control a tilt EQ that pivots around some mid frequency, simultaneously boosting highs/cutting lows or vice versa. This would effectively make the convolution output brighter or darker as a whole. Another useful pre-filter is a **band emphasis or**

notch if we want to reduce a particular resonant peak (though that might be advanced). We will at least include high-pass filtering of the excitation at an appropriate frequency (to remove DC or subsonic stuff that could waste headroom in the IR convolution). The **pre-EQ targets** likely refer to preset spectral shapes the user can dial in (for instance, “metallic bright” vs “soft muted” profiles). We can define a target curve and have the EQ morph toward it as a parameter is turned.

- **Post-EQ (Shaping Convolution Output):** After convolution, we also allow EQ on the **resulting audio**. This is useful for fine-tuning the output tone and dynamics. For instance, if the convolution IR has a very resonant low frequency ring that dominates, the user could dial it back with a low-shelf or parametric cut. Or if they want more “shimmer”, a high-shelf boost on the output can accentuate the tail’s highs. The design calls for *shelving/tilt* controls specifically, which implies broad tonal adjustments rather than surgical EQ (which would be more in an advanced panel). We might expose a “**Low/High Balance**” knob (which internally is a shelf filter boosting lows while cutting highs or vice versa), and a “**Overall Brightness**” knob (a high-shelf on output). These could overlap with the pre-tone control, so careful UI/UX is needed to not confuse the user. We can position one as affecting the nature of the strike (pre) and the other as affecting the tail/body (post).
- **Energy→Brightness Mapping:** A critical expressive feature is dynamically altering the tone based on energy. We will implement a mapping such that **higher energy makes the sound brighter and more chaotic**, following how real instruments behave ¹. In practice, this means as `E_global` rises, we increase the high-frequency content and maybe the amount of noise. We have a few mechanisms in place already: the resonators detune and coupling adds noise at high energy. In the convolution domain, we can simulate a similar effect: e.g., we could apply a **dynamic EQ or saturation** that kicks in when energy is high. One simple approach: when `E_global` exceeds a threshold, gradually mix in a **high-shelf boost** on the convolution output (making the overall sound brighter). We define a curve (perhaps an S-curve) that maps `E_global` to a dB boost value for the shelf. The “**chaos**” aspect could be interpreted as adding some distortion or additional noise at high energy. For instance, a slight bit of **wavefolding or overdrive** on the convolution output when loud can create higher harmonics and a rougher texture, simulating a crash’s biting quality. We could tie a **soft clipper’s drive** to `E_global` such that loud energies push the convolution output into gentle saturation. This is speculative but could yield a subtle effect of “everything getting a bit wild” when played hard. The mapping curves (energy→bright, energy→saturation) will be configurable in the advanced panel (e.g. thresholds and exponents). By default, it might be set so that at 50% energy we start to get noticeable brightening, and at 100% energy, maybe +6 dB at 8 kHz and a slight saturation.
- **Output Dynamics Control:** Finally, we ensure the overall output is controlled. With so many components (resonators + IR + noise), peaks could be high. We include an **output dynamics processor**, likely a limiter or fast compressor, primarily as a safety net. The user might have a knob for “**Master Dynamics**” or “**Compression**”, which essentially sets how aggressively the output is tamed. At 0 (no compression), the sound is fully dynamic – a hard hit could be much louder than a soft one. At higher settings, the plugin will auto-tame the peaks (perhaps targeting a more consistent level or adding sustain). This can be done with a simple one-knob compressor setting internally adjusting threshold/ratio (e.g. turning it might lower the threshold from 0 dB down to -12 dB and increase ratio from 1:1 to 4:1 progressively). The compressor would likely be post-sum (after adding resonator and convolution outputs). We’ll set it to have a fast attack to catch transients (to prevent digital clipping from sharp burst) and medium release to let the tail ring out. The **dynamics**

control not only protects from clipping but also lets the user shape the instrument's response – e.g. a heavily compressed setting can simulate that very controlled, pad-like gong sound where even soft hits bring out a lot of tail (because the gain is made up), while a no-compression setting is more realistic acoustically.

- **Mixing and Bleed:** By default, the output is the sum of resonator output and convolution output. We might give an **IR Level** knob if the user wants to emphasize or reduce the convolution component relative to the direct resonators. If set to 0, the plugin would act purely as a modal synth (which could sound more like a “dry” physical model). If set high, the full recorded IR sound dominates (more like sample convolution playback). A 50/50 mix might give a nice balance: the resonators add interactivity (detuning, coupling) which a static IR alone wouldn't have, and the IR adds richness beyond the 8 modes. This mix could be part of the core controls (e.g. call it “Body/Tone mix”).

With convolution integrated, we thus achieve a **hybrid architecture**: the resonators handle the immediate, interactive modal response, and the convolution handles the dense, realistic tail. The **character control** afforded by the EQ, brightness mapping, and dynamics allows the user to shape the output from a dark, mellow gong to a bright, harsh crash, and from a wide dynamic range to a compressed, effect-like sound – all within the same framework.

Parameter Mapping and Control Interface

We will expose around **10–15 core “knobs”** for high-level control, and provide an **“Advanced” panel** for tweaking deeper parameters (time constants, curve shapes, etc.). Below is a proposed mapping of parameters:

Core Parameters (Primary Controls):

These are the main controls a user can tweak on the front panel for broad changes in sound. Each maps to multiple internal settings as described.

1. **Strike Sensitivity:** Adjusts the input detection sensitivity for audio taps (and scaling for MIDI velocity).
2. *Purpose:* Calibrate how hard you need to hit to produce a loud sound. Turning it up means even light taps produce strong strikes.
3. *Default:* Medium (so normal playing yields a good range).
4. *Range:* Low (only very hard hits register) to High (extremely sensitive).
5. Internally, this sets the audio trigger threshold and velocity scaling curve.
6. **Mallet Hardness (Tone of Attack):** Controls the brightness of the strike's initial transient.
7. *Purpose:* Simulate different mallet types from soft (warm thump) to hard (sharp click).
8. *Default:* Middle setting (a moderately bright stick attack).
9. *Range:* Soft (low-pass filtered noise, little high-frequency content) to Hard (bright, almost click-like).
10. This adjusts the filter on the excitation burst and maybe the distribution of energy to high-frequency resonators.

11. **Decay (Global Damping):** Sets how quickly the sound dies out (overall ring time).
12. *Purpose:* Control sustain length of the instrument. Higher values = longer ring (like a large gong), lower = shorter (muted or smaller instrument).
13. *Default:* Relatively long (to showcase sustain, e.g. a few seconds global RT60).
14. *Range:* Short (1–2 s decay) to Long (10+ s).
15. Internally, this affects the global energy reservoir decay constant and the resonators' Q (damping) scaling. It might also scale the IR's tail length (could fade it out faster for shorter settings).
16. **Bright/Dark (Tone Tilt):** A macro EQ control to adjust overall tonal balance.
17. *Purpose:* Make the instrument sound brighter (emphasize highs) or darker (emphasize lows) uniformly.
18. *Default:* Neutral (flat EQ).
19. *Range:* Dark (-\$\$) to Bright (+\$\$) [in terms of shelving, e.g. ± 6 dB high-shelf].
20. This likely controls a tilt filter affecting convolution output (post-EQ) and possibly the excitation pre-EQ simultaneously for a cohesive shift. One knob could drive both a high-shelf boost in output and a slight cut on low-end in input, etc., giving a single "tone" feel.
21. **Chaos (Nonlinearity Amount):** Controls how much nonlinear behavior is introduced at high energies (detuning, mode coupling, noise).
22. *Purpose:* Dial between a more *pure, bell-like* response vs a *wild, crashing* response when hit hard.
23. *Default:* Medium-high (so the plugin demonstrates nonlinearity clearly on strong hits).
24. *Range:* Off/Linear (no extra detune or noise beyond linear ringing) to Extreme (strong detuning shifts, lots of noisy wash and mode coupling for aggressive chaos).
25. Internally, this scales the detune depth (D_i coefficients), the coupling coefficient k (or toggles coupling on/off beyond a threshold), and the intensity of the energy-driven noise burst. Essentially, it gates how pronounced those nonlinear enhancements are.
26. **Body/Tail Mix:** Balances the resonator output vs convolution IR output.
27. *Purpose:* Blend between synthesized modal sound and sampled IR richness.
28. *Default:* 50/50 or slightly favoring IR (for realism).
29. *Range:* 0 (All Resonators – dry model) to 100 (All IR – like convolution reverb of a real instrument)%.
30. Internally, simply a crossfade or gain mix between the two output components. At extremes, you might hear only the simplified modal tone or only the convolution (which could be a very detailed but possibly less dynamic sound).
31. **Size/Pitch:** Adjusts the fundamental tuning of the instrument (and perhaps perceived size).
32. *Purpose:* Let the user tune the instrument higher or lower. A larger gong typically has a lower pitch and longer decay; a smaller one is higher and shorter (though decay we already handle separately).
33. *Default:* 0 (nominal frequencies as designed).

34. *Range:* -12 to +12 semitones (for example) or a continuous percentage of frequency.
35. This multiplies all resonator frequencies by a constant and could also stretch the IR in time/pitch if feasible (though real-time IR pitch shifting is expensive; we might instead have multiple IRs or approximate via changing sample rate of convolution). If IR stretch is not done, this control mainly affects resonators, which is still useful. It effectively gives a *tuning* knob.
36. **Coupling Amount:** (If we decide to make this front-facing; could be advanced if too technical) Controls how strongly modes interact via coupling.
37. *Purpose:* Allows user to set whether the instrument is more “liner” vs “interactive” in its modes. A higher coupling leads to more unpredictable blooms and warbles when loud.
38. *Default:* Medium (some coupling audible on hard hits).
39. *Range:* 0 (off, modes independent) to 1 (max, heavy energy transfer).
40. Internally, scales the coupling matrix ψ or the fraction of energy swapped. If Chaos knob already controls coupling, we might not need this separate; we could tie coupling intensity to the Chaos parameter internally. In that case, we might replace this slot with another control, like “Excitation Depth” or “Roll Mode” if needed.
41. **Output Dynamics:** A one-knob master compressor/limiter control.
42. *Purpose:* Manage the dynamic range and punch. Lower settings = natural dynamics, higher = more compressed/sustained output.
43. *Default:* Off to mild (we assume user’s DAW can handle volume, but include for convenience and safety).
44. *Range:* 0 (no compression) to 100 (heavy limiting).
45. Internally, at 0 the limiter threshold is high or inactive; as you turn up, threshold lowers and maybe makeup gain increases slightly, so the overall loudness stays manageable. At max, the output is quite leveled, turning the instrument almost into a pad (swell). We will design it so that even at max, it doesn’t distort – it should just sound very contained.
46. **(Optional) Sustained Input Blend:** Since the user mentioned eventually supporting rubbing/mallet rolls as a separate system, we could reserve a knob for “**Sustain Input Blend**”.
 - *Purpose:* Crossfade between treating input as discrete hits vs a continuous excitation.
 - *Default:* 0 (since initially we won’t integrate that system).
 - *Range:* 0 (only strikes) to 100 (enable continuous excitation response).
 - This is speculative and might not be implemented in first version. It might remain in the advanced panel until the continuous input handling is ready.

These core controls give a performer quick access to the major dimensions of the sound: how sensitive/loud it is, how long it rings, how bright or chaotic it gets, and the mix of synthetic vs realistic tone.

Advanced Parameters (Expert Tweaks):

The advanced panel exposes all underlying settings for those who want to fine-tune or experiment beyond the normal range. This includes **time constants, exponents, filter frequencies, curve shapes, and safety limits**. We'll list them by subsystem:

• Hit Detection & Excitation:

- *Threshold (audio)*: The level above background that constitutes a hit. Default ~ -40 dB.
- *Retrigger time*: e.g. 50 ms default.
- *Velocity Curve*: Maybe an exponent for scaling audio amplitude to velocity. Default linear (1.0).
- *Excitation Envelope*: Attack/decay times for the burst. Defaults: attack 0–1 ms, decay ~30 ms.
- *Excitation Noise Color*: The cutoff frequency or filter coefficients for soft vs hard mallet extremes. Defaults tuned to a balanced bright thump (for example, low-pass at ~8 kHz for soft end, ~16 kHz for hard end).
- *Excitation Band Distribution*: An array of 8 gains (or relative weights) for how the burst feeds each resonator. Default could be all equal or based on a physical model (these might sum to 1 for energy conservation). Advanced user can tweak if they want a certain mode to be more/less excitable.

• Energy Accumulators:

- *Global Decay Time*: The 60 dB decay time for E_global. Default, say, 5 seconds.
- *Band Decay Times*: 8 values for each E_band or a formula. Defaults might decrease for higher bands (e.g. low band 5 s, high band 2 s).
- *Injection Gain (k)*: Scales how much energy is added per strike intensity. Default calibrated so a max velocity hit brings E_global near maybe 0.8 of E_max.
- *Injection Exponent (p)*: The power for nonlinear addition. Default ~0.5 (so diminishing returns).
- *E_max (Saturation Ceiling)*: A cap for normalized energy (e.g. 1.0). Could allow slight headroom >1 if you want some margin. By default 1.0 (100%).
- *Coupling Threshold (for energy envelope)*: If we decide coupling or secondary noise triggers when energy crosses a threshold. Default might be 0.6 of E_max.
- *Noise Injection Level*: A factor linking E_global to noise amplitude. Default such that at E_global=1.0, noise burst is, say, -6 dB relative to a full attack burst.

• Resonators (Modes):

- *Mode Frequencies*: f1...f8 in Hz. Defaults set to some inharmonic series (e.g. for a 200 Hz fundamental: 200, 380, 560, 760, 940, 1120, 1320, 1480 Hz – just example). Advanced users can retune to create different bell/gong sounds.
- *Mode Decay/Q*: Could specify either as Q factors or as decay time (time for amplitude to drop ~60 dB if excited alone). Defaults chosen per mode (lower modes might ring longer or shorter depending on material – often low modes ring longer for large gongs).
- *Mode Gain*: g1...g8 initial gain scaling for each mode's contribution. Defaults might be equal or slightly decreasing for higher modes (since typically higher partials might be quieter for a given strike).

- *Detune Depth*: D_i values or a global scale for detuning vs energy. For example, each mode might have D set such that at full energy, it detunes by X cents. Defaults could be small (a few cents for low modes, maybe up to 20 cents for some higher mode to get obvious beating).
- *Detune Curve Exponent*: If using E_{global}^n in detune formula – controls how nonlinearly detune responds. Default 1 or 2 for smooth onset.
- *Coupling Matrix*: If we allow editing, an 8x8 matrix (or sparse list) of coupling coefficients ψ_{mn} . Default all zero except some hand-picked pairs. E.g., could set coupling from a couple of high modes into the first low mode, and between adjacent modes perhaps. We might instead expose simpler controls like “High→Low coupling %” and “Mode spread coupling %” to populate this matrix. Defaults low (~10%).
- *Coupling Transfer Mode*: Perhaps an option of how to transfer (instantaneous vs over time). Likely fixed, but advanced could toggle the method if implemented (e.g. immediate injection vs gradual bleed).
- *Stereo Panning per Mode*: Possibly allow each mode an L/R position or phase offset. Default center or slight spread alternating. This is just for spatialization.

• **Convolution & EQ:**

- *Impulse Response File*: Ability to load a different IR. Default provided is presumably a gong IR. Advanced user can swap (e.g. to a different instrument's IR).
- *IR Length*: If we allow truncating or windowing the IR. Default is full length. They can reduce for shorter tails.
- *Pre-EQ Filter Coefficients*: Exact values for low-shelf, high-shelf used in Tone control. Defaults tuned to instrument. For instance, Low-shelf at 200 Hz ± 3 dB range, High-shelf at 5 kHz ± 6 dB range.
- *Post-EQ Shelving Freq*: The corner frequency for brightness control. Default say 6 kHz. Could allow adjusting if one wants brightness knob to target different area.
- *Tilt EQ Pivot*: The frequency around which the tilt rotates (maybe ~1 kHz by default).
- *Energy→Brightness Curve*: Possibly a small table or function. We might expose “Bright boost at max energy (dB)” default +6 dB, and “Brightness threshold (% energy)” default 50%, and maybe an “ease” parameter if using an S-curve.
- *Energy→Noise Curve*: Similarly, maybe an exponent or threshold for how noise scales. E.g., could be linear, or could be zero until 30% energy then ramp up. Defaults set by our earlier decisions.
- *Output Compressor Settings*: Threshold (default 0 dB), Ratio (default 1:∞ at max knob, down to 1:1 at min knob), Attack (default ~5 ms), Release (~200 ms), Knee (soft). The advanced panel can show these and allow tweaking if the one-knob isn't perfect for some users. For safety, we might set a hard limiter at 0 dB FS anyway which is not adjustable.

• **Safety Clamps and Misc:**

- *Max Resonator Amp*: A clamp on each resonator's amplitude to avoid numeric blow-up in extreme feedback cases. Likely we ensure filter stability rather than clamp audio, but just in case, an internal ceiling e.g. 10 dB above normal max. Not typically user-facing, but we can mention a “Clip Protection” switch (always on by default).
- *DC Blocking*: A toggle if needed (should be on by default to remove any DC offset from convolution or filters).

- **Oversampling:** Maybe allow oversampling if nonlinear wave-shaping (like saturation) is heavy. Default off or 2x if needed for anti-aliasing chaos.
- **Phase Modes:** Unlikely needed, but if someone wants all resonators to start phase-aligned on each hit vs free-running, we could allow a mode. Default free (sounds more natural as each hit doesn't phase-reset oscillators, avoiding a "same sound" each time).

All advanced parameters will have sensible defaults such that users who don't touch them get a convincing sound out-of-the-box. The core knobs will modulate several advanced parameters behind the scenes (we will document those mappings for clarity). For example, the **Chaos knob** might internally ramp up detune depth, coupling amount, and noise gain together; the **Bright/Dark knob** might adjust both pre and post EQ gains, etc. The advanced panel allows overriding or fine-tuning these relationships (e.g. maybe someone wants strong detune but no coupling – they could turn Chaos up but then manually turn coupling matrix entries to 0 in advanced).

Presets:

We will include a few factory presets as starting points, demonstrating the range of the instrument:

- **"Deep Gong"** – A large, dark gong sound. Uses long Decay, slightly lower Pitch, Dark tone, moderate Chaos (so some nonlinear bloom on hard hits). Coupling on to get that roar. Feels heavy and slow.
- **"Bright Crash Cymbal"** – Tuned higher, Decay a bit shorter, Bright tone, high Chaos (really splashy and chaotic at max hits), maybe slightly less low-end in IR mix. Good for loud accents.
- **"Bell-like"** – Emphasizes pure modal tones. Chaos off (no detune or noise), Brightness moderate, Decay medium. Sounds more like a steel bell or vibraphone bar – clean and not much tail (IR mix lower).
- **"Muted Impact"** – Very short Decay and high damping, making a dry percussive hit (like a muted gong or a frame drum-like thud). Possibly with a lower IR mix and strong compression so it's a tight hit with little ring.
- **"Evolving Roll (demo)"** – (For future continuous input integration) simulates a roll swelling: high Sensitivity, long Decay, Chaos high so that as it builds it gets very complex. This can showcase how multiple hits accumulate to a crescendo.

Each preset will adjust the core knobs (and some advanced as needed) to illustrate these distinct behaviors. Users can start from a preset and tweak core knobs to taste.

Extensibility: Sustained Input and Future Integration

(Note: The initial design focuses on percussive strikes, but the user expressed interest in supporting sustained excitations like rubbing or rolling a mallet continuously on the instrument. This poses different requirements, so designing that in parallel is wise.)

The current architecture could be extended to handle **continuous input** by essentially adding a second excitation path tailored for sustained energy transfer. For example, a "rubbing" excitation would involve continuous friction noise input instead of discrete bursts. The energy accumulation layer would then need to account for constant energy influx (perhaps driving the reservoirs toward a steady state rather than decaying to zero). The **excitation synthesis** for that case would bypass the sharp attack and instead feed a

smoother noise (e.g. filtered noise mimicking the friction) into resonators and convolution continuously. Likely, a separate set of parameters (friction noise gain, spectral tilt, etc.) would govern this mode.

However, combining these two in one model might be complex if done simultaneously; a practical approach is to have a **mode switch or blend** between *Strike Mode* and *Sustain Mode*. In strike mode (our current design), events trigger bursts. In sustain mode, the plugin could monitor the input for longer gestures (e.g. a rolling waveform) and respond differently – perhaps by opening a “sustain gate” that adds a constant excitation proportional to input amplitude. The **integration** of these two could mean, for instance, if a user is rolling a mallet (lots of fast repeated hits), the system could transition to treating it like a quasi-continuous drive (smoothing out individual transients into a sustained excitation).

For now, we will **design the strike-based system first**, ensuring it produces realistic accumulated effects for rolls (since multiple fast strikes naturally accumulate energy and noise in our model). That might already approximate a roll. When we explicitly design the continuous input handling, we will likely add: - A **separate friction noise exciter** that ramps up with continuous input. - Additional nonlinear behaviors (e.g. bowing a cymbal can induce different mode emphasis). - Possibly separate parameter tuning (the way coupling or detune works under continuous excitation might differ – e.g. chaotic turbulence might set in differently).

These details will be explored in a dedicated design once the strike system is proven. The architecture is modular enough to add another layer or branch for sustained excitation. The advanced panel might already have some “sustain mode” settings hidden, but we will expose them only once ready, to avoid confusion.

Conclusion

This build brief outlined a comprehensive design for a JUCE-based percussion synthesizer that hybridizes modal resonators with convolution. By layering a **hit detection** front-end, an **energy accumulation** system for handling multiple strikes, a flexible **excitation generator**, a bank of interactive **resonators** (with detuning and coupling for nonlinear richness), and a **convolutional tail** with detailed tone shaping, we achieve a playable, realistic instrument. The **parameter map** balances usability (core knobs for big changes) with depth (advanced parameters for precision), allowing both musicians and sound designers to benefit. The default presets will showcase the instrument’s ability to morph from a tame, bell-like chime to a fierce, crashing gong with just a few tweaks – all while ensuring that *repeated hits accumulate energy naturally*, as a real percussion instrument does ⁸. Future enhancements will incorporate continuous excitations for rubbing and rolling gestures, further broadening the expressive scope.

With this design, the implementation in JUCE can proceed: using its DSP modules for filters and convolution (e.g. `dsp::StateVariableFilter` for resonators, `dsp::Convolution` for IR) and the parameter system (`AudioParameterFloat` etc.) for the controls. The result will be a unique plugin instrument that combines the advantages of **physical modeling** (responsiveness and realism ⁵) with the flexibility of **sample-based convolution** (authentic timbre and complexity), mapped into an intuitive set of user parameters. The provided references and theoretical underpinnings give confidence that this architecture will capture the essence of nonlinear percussion behavior ¹ ¹³ while remaining efficient enough for real-time use.

Sources:

- Skare, T., & Abel, J. (2019). *Real-Time Modal Synthesis of Crash Cymbals with Nonlinear Approximations* – Notes the need for thousands of modes for cymbals and introduces modal coupling for nonlinear energy transfer ⁴ ¹³ .
- NESS Project (Bilbao et al.) – Describes nonlinear plate regimes: light strikes vs heavy strikes causing pitch changes, high-frequency bursts, subharmonics, and chaos ¹ . Highlights importance of nonlinear models for realism.
- Aimi, R. (2007). *Percussion Instruments Using Realtime Convolution* – Demonstrates using convolution with stored IR to augment a physical percussion controller, preserving nuance by using audio signals (not just triggers) to excite resonators ⁵ ⁶ .
- Reddit user discussion on cymbal realism – Emphasizes that real cymbal hits start from an ongoing vibration, not silence, underlining our accumulation approach ⁸ .
- Cirio, G. et al. (2018). – Research on wave turbulence in thin shells, explaining how energy spreads to high frequencies over time during sustained excitation ³ . (This concept influenced our energy→brightness mapping and noise injection.)
- Essl & Cook (1999/2004) – Banded waveguide synthesis method, showing efficient modeling of inharmonic percussion by band-splitting and allowing nonlinear interactions ²⁰ . Inspired our use of banded reservoirs and multi-resonator excitation.

¹ ⁹ ¹⁵ NESS - Next Generation Sound Synthesis

<http://www.ness.music.ed.ac.uk/archives/systems/cymbals-and-gongs>

² ⁴ ¹¹ ¹² ¹³ ¹⁴ ¹⁷ ¹⁸ ¹⁹ Real-Time Modal Synthesis of Crash Cymbals with Nonlinear Approximations, using a GPU

https://www.dafx.de/paper-archive/2019/DAFx2019_paper_48.pdf

³ ¹⁰ ¹⁶ (PDF) Percussion Synthesis Based on Models of Nonlinear Shell Vibration

https://www.researchgate.net/publication/224579888_Percussion_Synthesis_Based_on_Models_of_Nonlinear_Shell_Vibration

⁵ ⁶ ⁷ nime.org

https://www.nime.org/proceedings/2007/nime2007_154.pdf

⁸ Do you have any techniques for getting more organic cymbal sounds with drum machines? : r/WeAreTheMusicMakers

https://www.reddit.com/r/WeAreTheMusicMakers/comments/1052wei/do_you_have_any_techniques_for_getting_more/

²⁰ Banded waveguide synthesis - Wikipedia

https://en.wikipedia.org/wiki/Banded_waveguide_synthesis